

Informe Cientifico

Bloque III: El Techo de Rendimiento - CUDA GPU

Alumno: Aymen Bellakhal

Asignatura: Computacion de Altas Prestaciones (CAP)

Fecha de ejecucion del entorno: 16 de abril de 2026

GPU utilizada: NVIDIA Tesla T4, CUDA 13.0

Resumen

En esta practica se comparan implementaciones en CPU y GPU para operaciones con distintos perfiles de rendimiento: SAXPY, multiplicacion de matrices, conversion de imagen a escala de grises y generacion del fractal de Mandelbrot. El objetivo es identificar cuando la GPU compensa el coste de transferencia y cuando su paralelismo masivo permite superar ampliamente a la CPU.

Metodologia

Las pruebas se ejecutaron en Google Colab con una Tesla T4. En CUDA se usaron kernels con control explicito de limites, calentamiento previo y medicion mediante eventos CUDA cuando correspondia. El informe se ha construido con los resultados reales de ejecucion aportados en las capturas: tablas de SAXPY, grafica GEMM, procesamiento Lenna, Mandelbrot base y Mandelbrot con paletas personalizadas.

1. SAXPY: coste de arranque y control de limites

SAXPY calcula $Y = aX + Y$. Es una operacion muy paralelizable, pero con baja intensidad aritmetica: por cada elemento se realizan pocas operaciones y se mueve memoria. Por ello, en tamanos pequenos el coste fijo de lanzar el kernel y transferir datos puede ocultar la ventaja de la GPU.

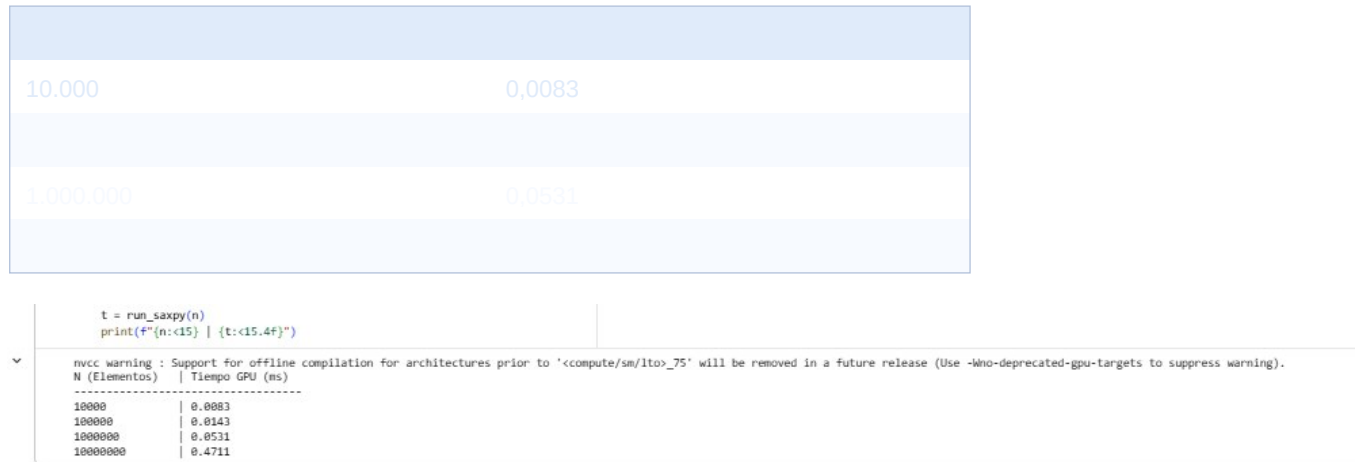


Figura 0. Captura de la ejecucion real de SAXPY con los tiempos obtenidos en CUDA.

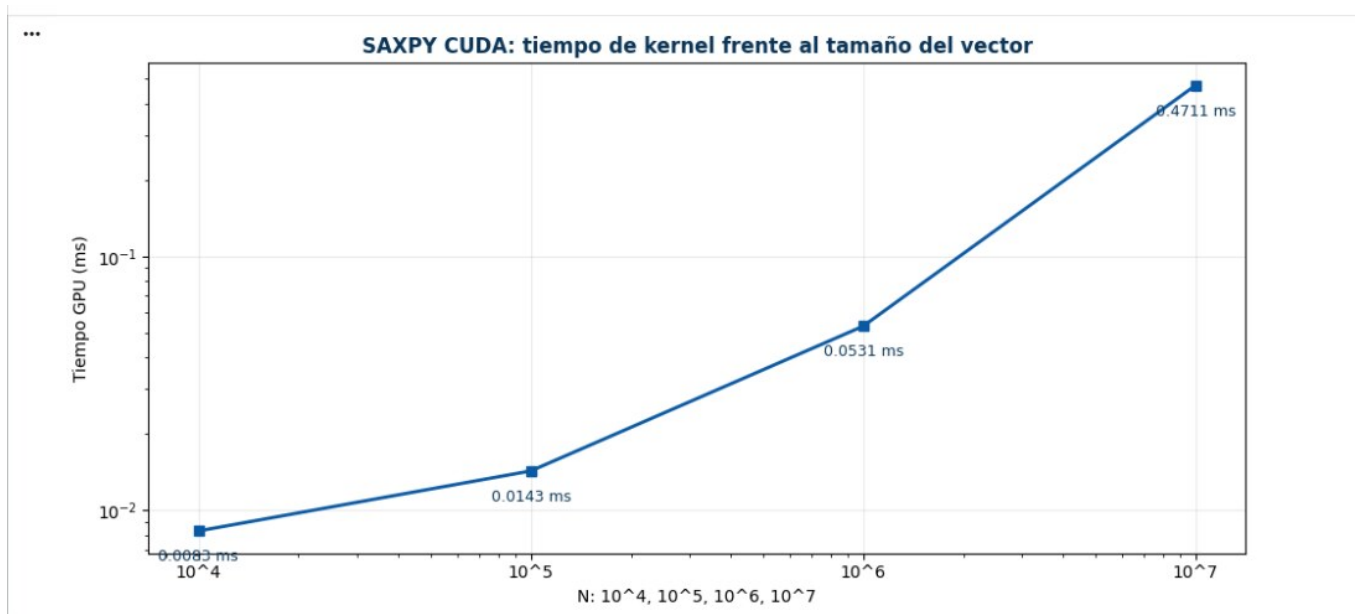


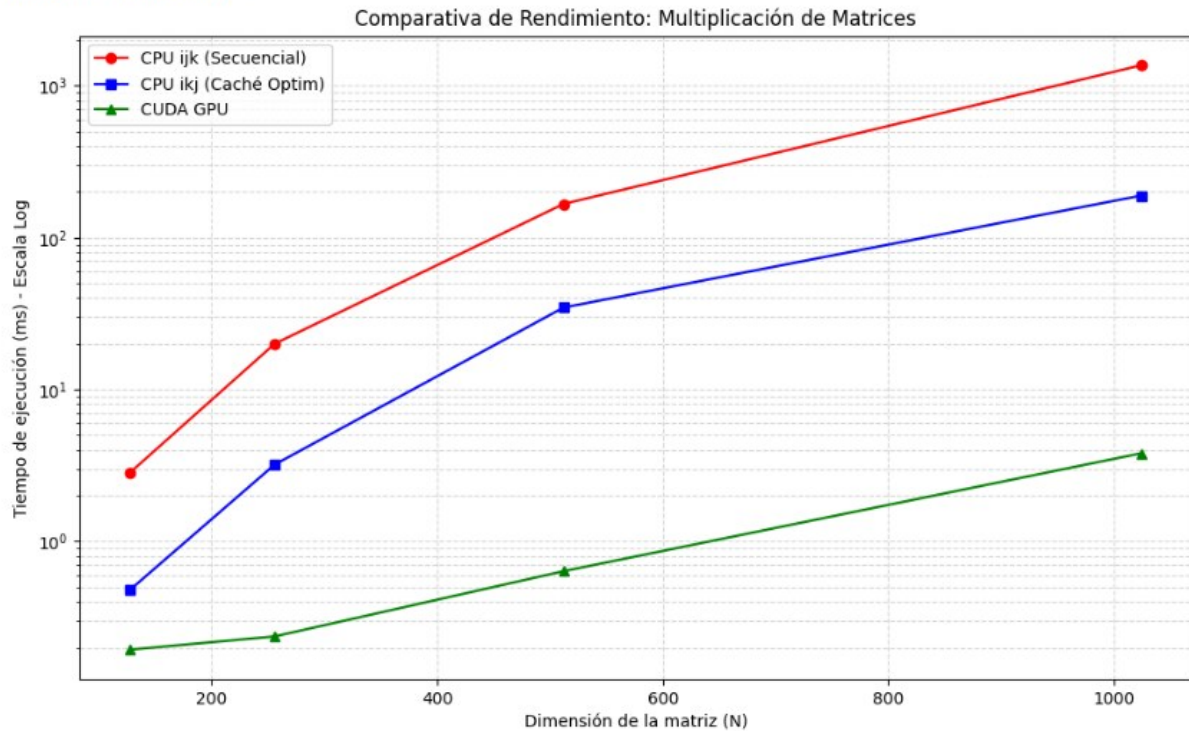
Figura 1. Grafica real generada en el notebook para SAXPY CUDA: tiempo del kernel frente al tamaño del vector.

El kernel usa $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ y la condicion $\text{if} (i < n)$. Esta condicion es obligatoria porque se lanzan bloques de 256 hilos y el numero total de hilos suele superar el tamaño real del vector. Sin ella, los hilos sobrantes escribirían fuera del array.

2. Multiplicación de matrices (GEMM)

Se compararon tres versiones: CPU ijk, CPU ikj y CUDA GPU. La versión ikj mejora mucho en CPU porque hace que el bucle interno acceda de manera secuencial a B y C, aprovechando las líneas de cache. En cambio, ijk provoca saltos grandes al recorrer columnas de B y genera muchos fallos de cache.

```
Iniciando Benchmark...
Probando N = 128... ✓ OK
Probando N = 256... ✓ OK
Probando N = 512... ✓ OK
Probando N = 1024... ✓ OK
```



Resultados para N=1024:
🚀 Speedup CPU ikj vs ijk: 7.21x
🔥 Speedup GPU vs CPU ijk: 358.34x

Figura 2. Comparativa de rendimiento GEMM en escala logarítmica según la ejecución real. Para $N=1024$ se obtuvo speedup CPU ikj vs ijk de 7,1x y speedup GPU vs CPU ijk de 358,34x.

La gráfica muestra que la GPU mantiene una curva mucho más baja. Aunque la complejidad teórica sigue siendo $O(N^3)$, el paralelismo masivo reduce enormemente la constante de tiempo efectiva.

3. Caso visual: Lenna en escala de grises

La conversión a escala de grises trata la imagen como una matriz de píxeles RGB. Cada píxel se puede calcular de forma independiente, por lo que el problema encaja bien en CUDA. No obstante, sigue siendo una operación relativamente ligera, por lo que el speedup es menor que en Mandelbrot.

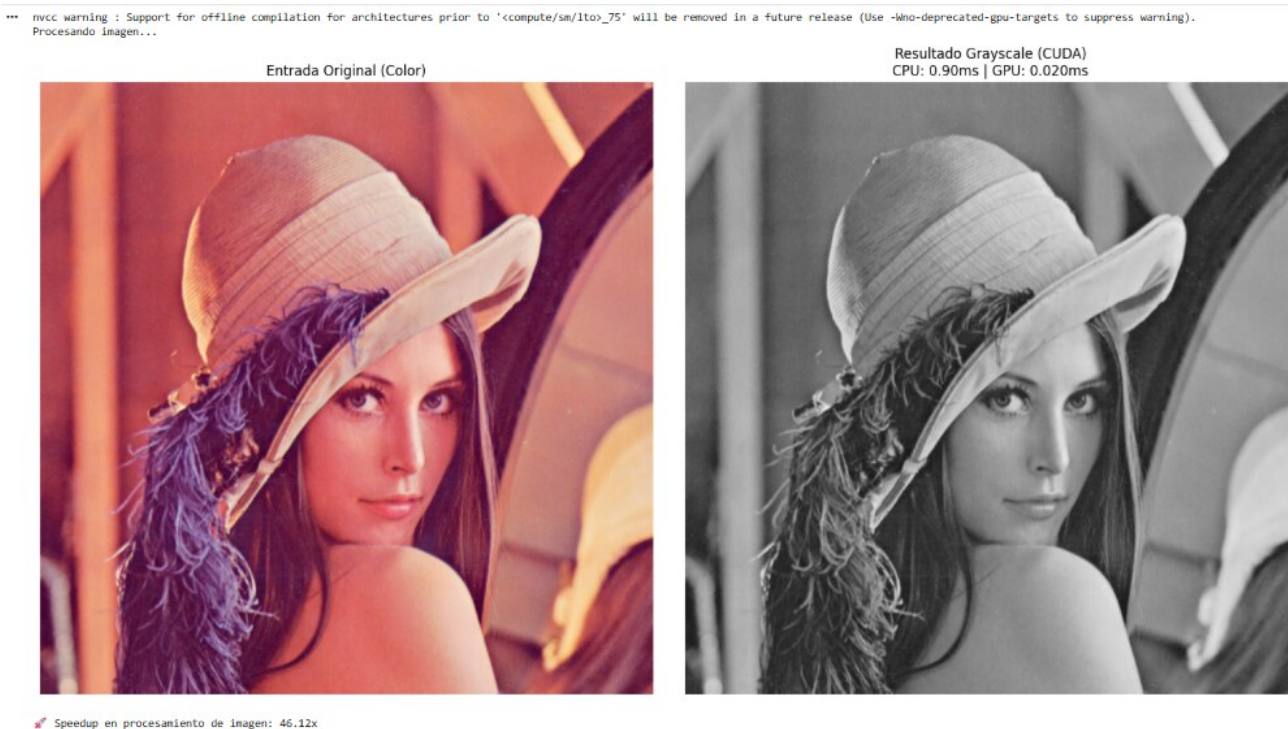


Figura 3. Procesamiento de imagen Lenna. CPU: 0,90 ms; GPU: 0,020 ms; speedup: 46,12x.

4. Mandelbrot: computo intensivo

Mandelbrot presenta una intensidad aritmética mucho mayor: cada píxel puede requerir cientos de iteraciones con multiplicaciones y sumas complejas antes de escribir unos pocos bytes de salida. Por ello, la GPU aprovecha mejor sus unidades aritméticas.

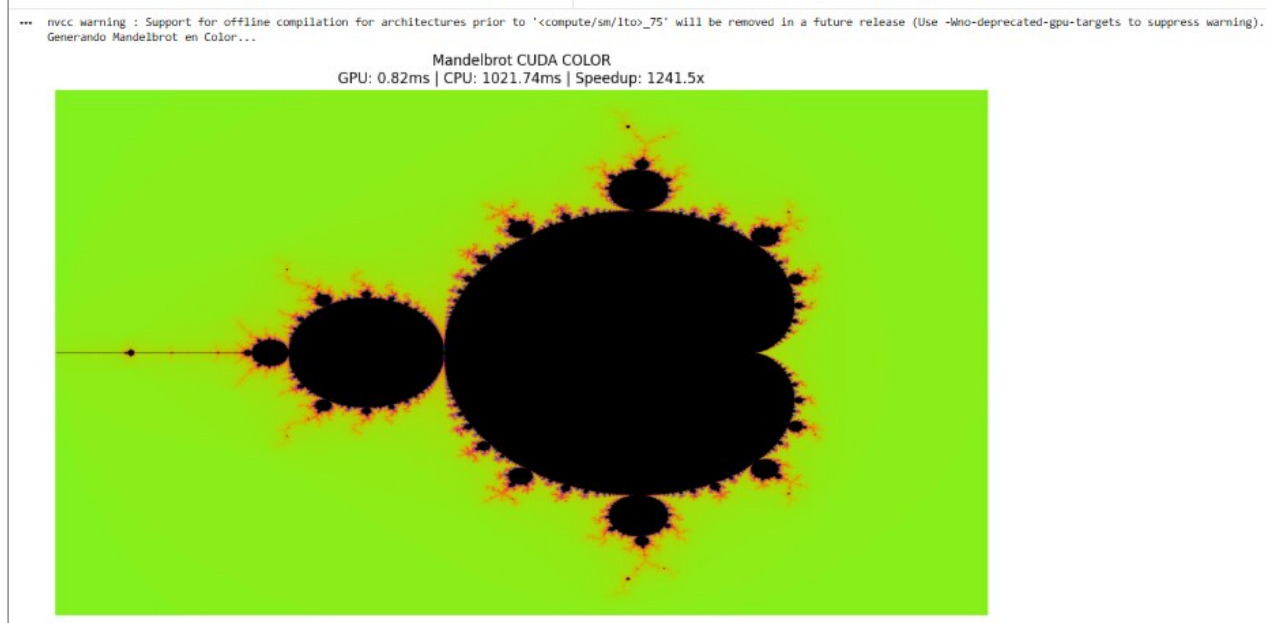


Figura 4. Mandelbrot CUDA color. GPU: 0,82 ms; CPU: 1021,74 ms; speedup: 1241,5x.

5. Reto creativo: paletas personalizadas

Se modifico la funcion de color usando funciones seno y coseno para generar dos paletas: Fuego y Cyberpunk. La ejecucion mantuvo tiempos practicamente iguales a la version base, lo que indica que las unidades de calculo de la GPU procesan funciones matematicas complejas de manera muy eficiente y paralela.

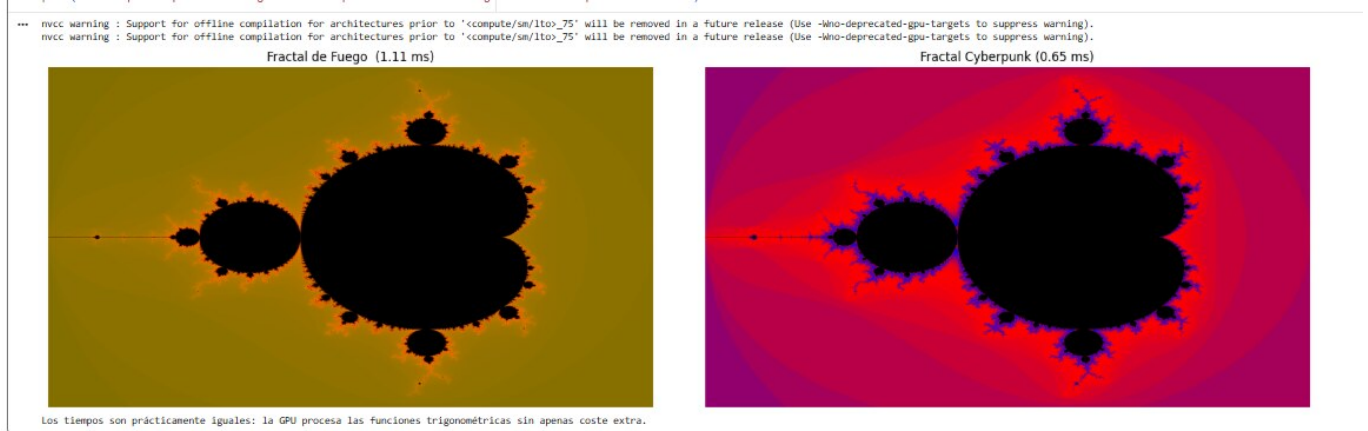


Figura 5. Captura del fractal de Mandelbrot con paletas personalizadas: Fuego (1,11 ms) y Cyberpunk (0,65 ms).

Esta figura cumple el requisito del reto creativo: incluir al menos una captura del fractal con paleta personalizada.

6. Respuestas razonadas al cuestionario

I. SAXPY y control de hilos

1. En nuestro resultado de SAXPY, el kernel CUDA tarda solo 0,0083 ms para 10^4 elementos, pero ese dato mide principalmente el tiempo del kernel, no todo el coste real de usar la GPU desde cero. Para un vector pequeño, la operación $Y = aX + Y$ hace muy poco cálculo por elemento: básicamente una multiplicación y una suma. Si se incluye el coste completo de preparar la ejecución, copiar datos entre la RAM de la CPU y la memoria de la GPU por PCI-Express, lanzar el kernel y sincronizar, la CPU puede ser igual o incluso más rápida. Ese coste fijo se llama overhead: se paga casi igual aunque el problema sea pequeño. En cambio, cuando pasamos a 10^7 elementos, el tiempo del kernel sube a 0,4711 ms, pero ahí ya hay suficiente trabajo paralelo para que la GPU empiece a compensar mucho mejor.

2. La condición $\text{if } (i < n)$ evita que los hilos sobrantes trabajen sobre posiciones que no existen. Por ejemplo, si se lanza un bloque de 256 hilos para procesar un vector de 200 elementos, los hilos con índices 200 a 255 también calcularían una posición. Sin la condición, esos 56 hilos accederían fuera del vector y escribirían en memoria global de la GPU que no pertenece a ese array. Podrían modificar datos de otro vector, memoria reservada por CUDA o zonas no válidas. El resultado podría ser una salida corrupta, errores difíciles de detectar o un fallo del kernel. Por eso el control de límites es obligatorio cuando el número de hilos lanzados no coincide exactamente con el tamaño del problema.

II. GEMM

3. En nuestros resultados de GEMM, cambiar de CPU ijk a CPU ikj produjo un speedup de 7,1x para $N=1024$. La razón no es que se hagan menos operaciones: ambas versiones calculan la misma multiplicación y siguen teniendo complejidad $O(N^3)$. La mejora viene del acceso a memoria. En ijk , el bucle interno accede a $B[k*N+j]$, es decir, recorre una columna de B saltando de fila en fila; esos saltos grandes provocan fallos de cache porque la CPU carga líneas de memoria que apenas reutiliza. En ikj , el bucle interno recorre $B[k*N+j]$ y $C[i*N+j]$ de forma contigua, aprovechando mucho mejor la cache. En GPU, probar ijk e ikj de la misma forma no tiene tanto sentido: CUDA no se optimiza solo cambiando el orden de bucles, sino organizando hilos, bloques, coalescencia de accesos y, para una versión avanzada, memoria compartida. La versión GPU de la práctica lanza muchos hilos en paralelo, uno por celda de C , por eso la comparación importante es CPU frente a GPU.

4. En la gráfica logarítmica de GEMM, la línea verde de CUDA permanece muy por debajo de las dos líneas de CPU. Para $N=1024$, la GPU consigue un speedup de 358,34x frente a CPU ijk , lo que confirma que al aumentar el tamaño del problema la GPU escala mucho mejor. Todas las versiones realizan muchas operaciones, pero la CPU las ejecuta con pocos núcleos y sufre más por cache y memoria. La GPU reparte el cálculo de miles de celdas entre muchísimos hilos, reduciendo mucho el tiempo total. Esto significa que cuanto más masiva sea la matriz, más rentable resulta usar GPU, siempre que el problema tenga paralelismo suficiente.

III. Mandelbrot: computo intensivo

5. En Mandelbrot obtuvimos GPU = 0,82 ms, CPU = 1021,74 ms y un speedup de 1241,5x, muy superior al de SAXPY. La explicación es la intensidad aritmética. SAXPY mueve varios datos de memoria para hacer solo dos operaciones por elemento, por lo que está limitado por memoria y por transferencias.

Mandelbrot, en cambio, calcula para cada píxel muchas iteraciones de una fórmula compleja antes de escribir el color final. Eso significa muchos cálculos por muy pocos bytes escritos. Es un caso compute-bound: el cuello de botella principal es el cálculo, justo donde la GPU destaca por tener miles de unidades aritméticas trabajando en paralelo.

6. Aunque el fractal parezca orgánico, no es aleatorio. Cada píxel de la imagen corresponde a un punto concreto C del plano complejo, definido por su posición en una rejilla fija de 1920×1080 . A partir de ese C se aplica siempre la misma fórmula $Z(n+1) = Z(n)^2 + C$ durante un número máximo de iteraciones. Como no hay azar ni comunicación entre píxeles, cada hilo calcula una función pura: mismo píxel, mismo C , mismo resultado. Por eso todos los alumnos obtienen la misma forma del conjunto de Mandelbrot si usan los mismos límites, resolución e iteraciones; lo único que cambia puede ser la paleta de colores.

IV. Reflexión final y color

7. Con la paleta base de Mandelbrot obtuvimos 0,82 ms en GPU. Al modificar `get_color`, la variante Fuego tardó 1,11 ms y la Cyberpunk 0,65 ms. No se observa una penalización grande ni constante por usar funciones seno/coseno: Fuego sale algo más lenta que la base, pero Cyberpunk incluso aparece más rápida, probablemente por variabilidad de medición y pequeñas diferencias del código. La conclusión razonable es que añadir trigonometría al color no destruye el rendimiento. Las GPUs modernas tienen muchas ALUs y unidades especializadas para funciones matemáticas, de modo que pueden evaluar estas operaciones en paralelo para millones de píxeles con un coste muy bajo comparado con hacerlo en CPU.

8. Según nuestros resultados, movería un algoritmo a GPU cuando tenga tres características: mucho tamaño, mucho paralelismo y bastante cálculo por dato. GEMM es un buen ejemplo: para $N=1024$ obtuvimos 358,34x frente a CPU ijk. Mandelbrot es aún más claro: 1241,5x porque cada píxel requiere muchas operaciones independientes. También merece la pena en imagen cuando hay muchos píxeles independientes, como Lenna, donde vimos 46,12x. En cambio, no lo recomendaría para problemas pequeños, tareas con pocas operaciones por dato como SAXPY pequeño, algoritmos muy secuenciales o casos donde haya que copiar datos continuamente entre CPU y GPU. En esas situaciones, el overhead puede comerse la mejora del paralelismo.

7. Conclusion

La practica confirma que la GPU no es automaticamente mas rapida en todos los casos: necesita suficiente trabajo para amortizar su overhead. En SAXPY, el beneficio esta limitado por el ancho de banda y la transferencia. En GEMM y Mandelbrot, al aumentar el calculo por dato movido, la GPU muestra su verdadero potencial. La version CPU ikj demuestra tambien que optimizar memoria/cache puede producir mejoras importantes incluso sin cambiar de hardware.

La conclusion principal es que la decision CPU vs GPU debe basarse en el perfil del algoritmo: paralelismo disponible, tamano del problema, intensidad aritmetica, regularidad de memoria y coste de transferencia. CUDA es especialmente recomendable cuando miles o millones de tareas independientes pueden ejecutarse a la vez.

Archivo de entrega: Bellakhal_AymenBloque3.pdf